Sanskrit Document Retrieval-Augmented Generation (RAG) System

Author: Aditya Pethe

---

1. Introduction

Retrieval-Augmented Generation (RAG) combines information retrieval with language generation to produce accurate and context-aware responses. Instead of relying only on pre-trained knowledge, the system retrieves relevant information from documents and uses it to generate answers.

The objective of this project is to design and implement a CPU-based RAG system capable of processing Sanskrit documents and answering user queries efficiently.

---

2. System Architecture

The system follows a modular RAG architecture consisting of:

- **Retriever**: Retrieves relevant document chunks
- **Generator**: Generates answers using retrieved context

Workflow:

Query → Embedding → FAISS Retrieval → Top-K Chunks → Context + Query → LLM → Answer

This modular design ensures scalability, clarity, and adherence to RAG principles.

---

3. Data and Preprocessing

Data

- Sanskrit text (Devanagari script)
- English and mixed-language inputs

Preprocessing Steps

- Text normalization
- Sentence segmentation
- Tokenization
- Chunking (splitting text into smaller segments)

Chunking improves semantic retrieval accuracy.

---

4. Retrieval Mechanism

Embeddings

- Multilingual Sentence Transformer

- Converts text into dense vector representations

Vector Store

- FAISS used for efficient similarity search

Retrieval Process

- Query converted to embedding
- Cosine similarity computed
- Top-K relevant chunks retrieved

This enables fast and scalable retrieval on CPU.

---

5. Generation Mechanism

Model

- FLAN-T5 (google/flan-t5-small)

Approach

- Combine query and retrieved context
- Provide structured instruction to model
- Generate concise answer (2–3 lines)

The system focuses on answering the question rather than summarizing the entire text.

---

6. Functionality Validation

The system successfully performs end-to-end RAG operations:

- Accepts user queries
- Retrieves relevant document chunks
- Generates context-aware answers

Example:

Query: Who is Kalidasa?
Output: Kalidasa is a famous Sanskrit poet and playwright known for his literary works.

This demonstrates correct retrieval and generation behavior.

---

7. CPU Optimization

- Entire system runs on CPU (no GPU required)
- Lightweight models used (MiniLM, FLAN-T5-small)

- FAISS ensures fast vector search
- Efficient chunking reduces computation overhead

This ensures low latency and efficient inference.

---

8. Implementation Details

Backend

- FastAPI used for API development

Frontend

- Streamlit used for user interface

Pipeline

- Ingestion → Preprocessing → Embedding → Storage
- Query → Retrieval → Generation → Response

The system is modular and easy to extend.

---

9. Code Quality and Modularity

- Clear separation of components (processing, retrieval, generation)
- Modular design for scalability
- Clean and readable code structure
- Reproducible setup via requirements.txt

---

10. Challenges

- Limited Sanskrit embedding support
- Mixed-language handling
- Lower similarity scores for Sanskrit queries

---

11. Limitations

- Sanskrit embeddings are not highly optimized
- Better performance with English/mixed queries
- Limited understanding of complex Sanskrit grammar

---

12. Future Improvements

- Domain-specific Sanskrit embeddings
- Translation layer integration
- Hybrid retrieval (keyword + semantic)
- More advanced LLM integration

---

13. Conclusion

This project demonstrates a complete CPU-based RAG system for Sanskrit documents. It highlights modular design, efficient retrieval, and practical handling of low-resource language challenges.